E-COMMERCE SALES MANAGEMENT

SQL project

Description: **The E-Commerce Sales database is designed to store, manage, and analysis data related to customers, products, orders, and sales transaction .This system help track customer purchase, product performance, revenue trends and overall business insight.**

Tables in the Database

1.Customers table- **store customer details such as customer id, name, email, and contact information.**

2.Products table-Contains product information including product id, name, category, price and stock.

3.Orders tables- **Stores order details such as order id, customer id, and order date.**

4.Order items table- **Track products in each order including quantity and item price.**

5.Payments table-Records payment detail like payment id, order id, amount, and payment mode.

Queries:

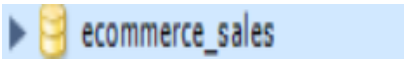
Create database

Question :- Creating Database

Input Query:

```
create database Ecommerce_sales;
```

Output:



Insight : A Separate Database is created to manage library operations.

CREATE TABLE : Create a table of customers where there are customer id, name, email, phone no and city available with appropriate constraints.

Input Query :

```
-- customers table
create table customers(customer_id int primary key,
customer_name varchar(50) not null,
customer_email varchar(100)unique,
customer_city varchar(30));
```

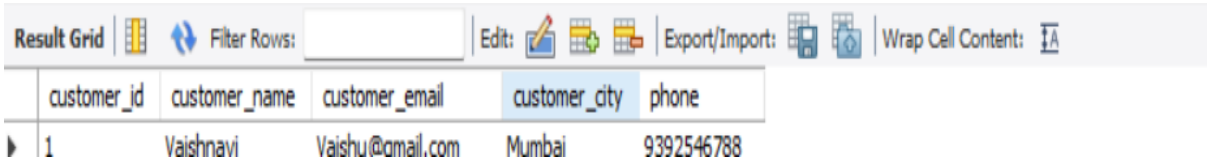
Output:

Insight : Defines primary keys and Unique constraints prevents duplicate records, while Not Null ensures essential identity data is captured.

Inserting values :

Question: Inserting values in each column.

Input Query:



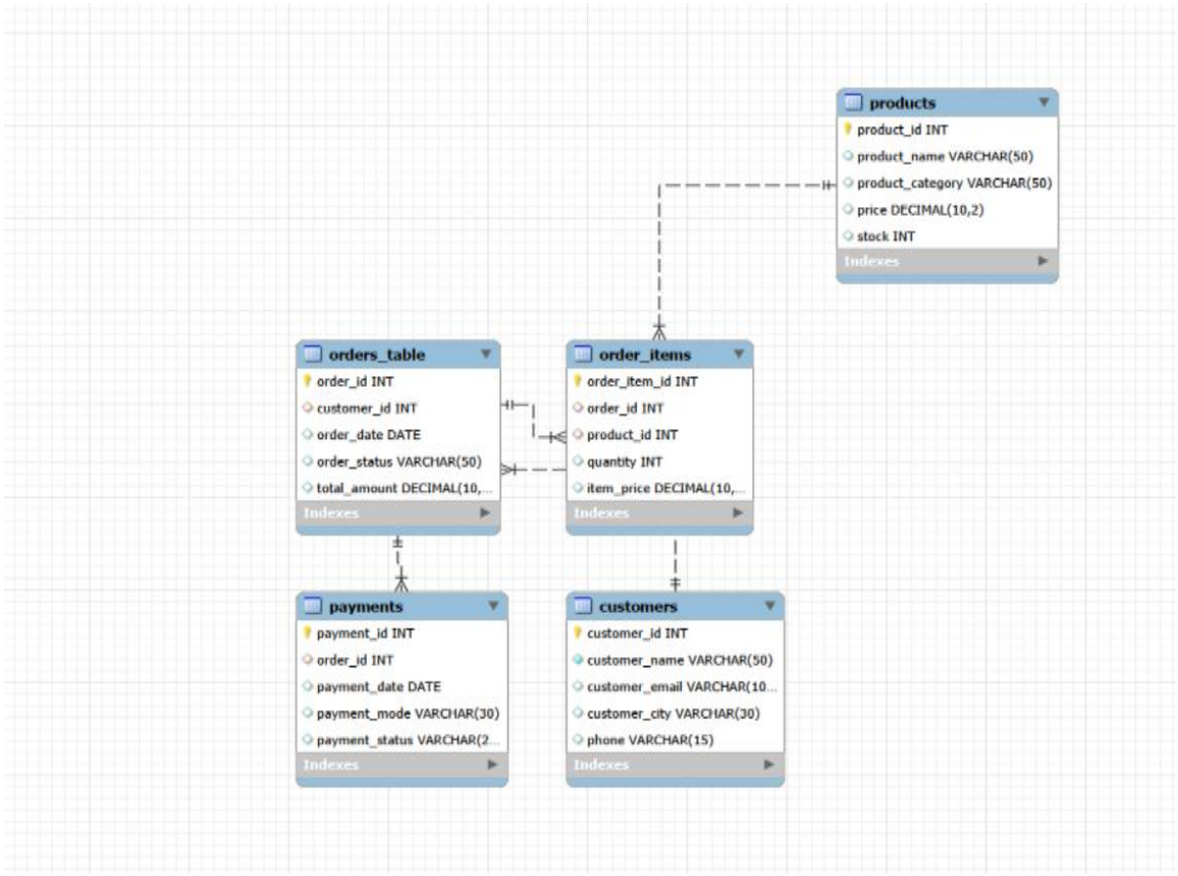
customer_id	customer_name	customer_email	customer_city	phone
1	Vaishnavi	Vaishu@gmail.com	Mumbai	9392546788

Output:

customer_id	customer_name	customer_email	customer_city	phone
1	Vaishnavi	Vaishu@gmail.com	Mumbai	9392546788
2	Sameer	Sameer@gmail.com	Pune	9987345678
3	Aashish	Aashi@gmail.com	Locknow	9876549266
4	Raj	Raj@gamil.com	Dehli	7729997648
5	Malti	Malti@gmail.com	Nashik	8796886567
6	Ruchi	Ruchi@gmail.com	Nagpur	7689657765
7	Laxman	Laxman@gmail.com	Bhopal	7656898976
8	Bunty	Bunty@gmail.com	Surat	9594678876
9	Kapil	Kapil@gmail.com	Gujarat	9293456777
10	Chandan	Chandan@gmail.com	Jaipur	8897764567
11	Akash	Akash@gmail.com	Kashi	9697878654
12	Sanjay	Sanjay@gmail.com	Mathura	7867987665
13	Atul	Atul@gmail.com	Ayodhya	8767568794
14	Karam	Karam@gmail.com	Thane	8897676578
15	Shruti	Shruti@gmail.com	Mumbai	7711668976

Insight : Populates the tables with sample data, which is essential for testing, developing, and demonstrating the functionality of all other queries.

ENTITY RELATIONSHIP DIAGRAM



Update

Question: Update the order _status to “Pending” for the order whose order _id =201.

Input Query

```
-- Update (DML)
Update orders set order_status = 'Pending'
where order_id = '201';
```

Output:

order_id	customer_id	order_date	order_status	total_amount
201	1	2024-01-01	Pending	30000.00
202	2	2024-02-03	Delivered	65000.00
203	3	2024-02-07	Pending	3000.00
204	4	2024-03-06	Delivered	2000.00
205	5	2024-04-12	Cancelled	5000.00
206	6	2024-05-23	pending	45000.00
207	7	2024-06-06	Deliered	23000.00
208	8	2024-07-14	Cacelled	1500.00
209	9	2024-08-02	Pending	34000.00
210	10	2024-09-26	Delivered	18000.00
NULL	NULL	NULL	NULL	NULL

Insight: Update is used to modify existing data in a table.

Window function:

Question: finding total amount.

Input Query

```
select count(*)as total_amount from orders;
```

Output:

	total_amount
▶	10

Insight: The query count(*), so it is counting rows, not calculating a monetary total.

Question: Assign order number to each customer base on order date.

Input Query

```
select customer_id,order_id,order_date,
row_number() over (partition by customer_id order by order_date)
as order_no from orders;
```

Output:

	customer_id	order_id	order_date	order_no
▶	1	201	2024-01-01	1
	2	202	2024-02-03	1
	3	203	2024-02-07	1
	4	204	2024-03-06	1
	5	205	2024-04-12	1
	6	206	2024-05-23	1
	7	207	2024-06-06	1
	8	208	2024-07-14	1
	9	209	2024-08-02	1
	10	210	2024-09-26	1

Insight: Used in customer order history. Give unique number per row.

Rank () Query

Question: Find products sales ranking.

Input Query

```
-- 2 find product sales ranking
select product_id, sum(quantity)AS total_sold,
Rank() over(order by sum(quantity)desc)AS rank_no from order_items
group by product_id;
```

Output:

product_id	total_sold	rank_no
104	3	1
102	2	2
105	2	2
107	2	2
109	2	2
101	1	6
103	1	6
106	1	6
108	1	6
110	1	6

Insight: Sum(quantity)calculate total sales per products.

Aggregate Function:

Question: Calculate total sales amount

Input Query:

```
select sum(quantity * item_price) AS total_sales from order_items;
```

Output:

total_sales
357500.00

Insight: Measure total revenue.

Limit and offset function:

Question: Find 2nd highest priced products.

Input Query:

```
select * from products order by price desc limit 1 offset 1;
```

Output:

product_id	product_name	product_category	price	stock
106	LED TV	Sony	45000.00	10
NULL	NULL	NULL	NULL	NULL

Insight: Ignores the maximum price on the next highest values.

Date:

Question: find today date.

Input Query

```
select current_date();
```

Output:

	current_date()
▶	2025-12-24

Insight: Useful for date _based filtering and calculations.

Question: Find the month of each orders.

Input Query

```
select order_id, month(order_date) as order_month from orders;
```

Output:

	order_id	order_month
▶	201	1
	202	2
	203	2
	204	3
	205	4
	206	5
	207	6
	208	7
	209	8
	210	9

Insight: Month (order -date) extracts the month number from each order date, helping analysis.

JOINS

Question: Find customer name with order details.

Input Query

```
-- customer name with order details
select c.customer_name, o.order_id,o.order_date
from customers c
inner join orders o on c.customer_id = o.customer_id;
```

Output

customer_name	order_id	order_date
Vaishnavi	201	2024-01-01
Sameer	202	2024-02-03
Aashish	203	2024-02-07
Raj	204	2024-03-06
Malti	205	2024-04-12
Ruchi	206	2024-05-23
Laxman	207	2024-06-06
Bunty	208	2024-07-14
Kapil	209	2024-08-02
Chandan	210	2024-09-26

Insight: Joining customers and orders shows each customer's name with their order details.

Subqueries:

Question: Second Highest Price From Products.

Input Query:

```
select * from products where price = (select min(price) from products);
```

```
select * from products where price = (select max(price) from products);
```

Output:

product_id	product_name	product_category	price	stock
102	Laptop	Lenovo	65000.00	20
NULL	NULL	NULL	NULL	NULL

product_id	product_name	product_category	price	stock
108	Keyboard K1	HP	1500.00	12
NULL	NULL	NULL	NULL	NULL

Insight: Subqueries identify the highest and lowest price products by comparing prices against max () and min () values.